



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information

Fall Semester 2026-2027

Department of Information Technology

BITE497J - Project – I

Review -1

Date: 19.8.2026

**VITian Chatbot: A Multi-Agent LLM-Based Conversational
Assistant for Academic, Placement, and Career Guidance**

Submitted by

Ashvin Mavila Kannothe

23BIT0152

Kshitiz Khandelwal

23BIT0121

Sermin Singh

23BIT0344

Under the Guidance of

Dr. Chemmalar Selvi G
Assistant Professor Sr. Grade 2

Table of Contents

S.No.	Contents	Page No.
	Abstract	iii
1	Introduction	4
2	Problem Statement	5
3	Objectives	6
4	Scope of the Project	7
5	Proposed System	8
6	Literature Survey	9
7	Methodology	13
8	Software Requirements	21
9	System Architecture	22
10	UML Diagrams	23
11	Summary	26
12	References	27

ABSTRACT

The rapid growth of academic, placement, and campus-related information at large technical universities has made it increasingly difficult for students to track exam schedules, recruitment deadlines, and eligibility criteria that are otherwise scattered across university portals and unstructured placement emails. Existing helpdesk and FAQ-based systems retrieve static responses from predefined rule sets, offering narrow coverage of student-specific queries and no means of tracking a student's placement preparation over time. Current placement communication is further complicated by its email-based, unstructured nature, which requires students to manually parse notices for deadlines and eligibility conditions, often resulting in missed opportunities. General-purpose large language model chatbots address the coverage problem but remain ungrounded in verified institutional data and cannot orchestrate multi-step preparation activities such as skill-gap analysis or reminder scheduling. The proposed system, VITian Chatbot, introduces a multi-agent conversational assistant built on AWS Lex V2 for natural language understanding, coordinated by a Supervisor LLM agent implemented using LangGraph that delegates tasks to specialised Company Research, Planner, Progress, and Notification agents. AWS Lambda parses incoming placement emails into structured JSON stored in Amazon S3, while a FastAPI backend orchestrates retrieval-augmented responses drawn from a PostgreSQL and ChromaDB knowledge base. Amazon EventBridge and SNS automate daily reminders, and IAM together with CloudWatch secure and monitor the pipeline. By combining grounded retrieval with autonomous multi-agent planning, the system evolves from a static FAQ bot into an adaptive career-preparation mentor capable of gap analysis, personalised study roadmaps, thereby improving placement readiness and reducing missed deadlines for VIT students.

1.INTRODUCTION

Large technical universities generate a constant stream of academic and placement-related information — exam notices, recruitment drives, eligibility updates, and interview schedules — that students are expected to track on their own. In practice this information is scattered across portals, circulars, and email threads, and it changes often enough that a student can miss an opportunity simply because a notice was buried in an inbox. Most campus chatbots today are built to answer a narrow set of frequently asked questions and stop there; they do not help a student plan preparation, monitor progress, or get reminded before a deadline.

At the same time, general-purpose LLM chatbots such as ChatGPT or Gemini can hold a much wider conversation, but they are not grounded in the specific, fast-changing information a placement season actually needs — which companies are visiting, what the eligibility criteria are this year, or what a particular recruiter's interview process looks like. Left ungrounded, these models tend to answer confidently even when they are wrong, which is a serious problem in a context where a wrong eligibility answer can cost a student a chance to apply.

VITian Chatbot is our attempt to combine the two strengths — the conversational flexibility of an LLM and the reliability of grounded, institution-specific data — inside a multi-agent architecture. Instead of one model trying to do everything, a Supervisor agent built with LangGraph routes each request to a specialised agent: a Company Research Agent that answers placement questions using retrieval-augmented generation, a Planner Agent that builds and revises study roadmaps, a Progress Agent that tracks preparation, and a Notification Agent that sends timely reminders. This report documents the first review stage of the project, covering the problem definition, related work, proposed methodology, and the experimental framework we will use to evaluate whether this design actually delivers on its claims.

2.PROBLEM STATEMENT

Students at large technical universities do not have a single, reliable assistant that can (a) answer placement and company-specific questions with verified, up-to-date information, (b) build and adapt a personalised preparation plan based on the student's actual progress, and (c) proactively notify them before deadlines. Rule-based FAQ bots are reliable but rigid and cannot answer anything outside their scripted intents. General-purpose LLM chatbots are flexible but ungrounded, which makes them prone to hallucinating facts about companies, eligibility, or dates. Neither category tracks a student's preparation over time or closes the loop between what a student struggles with and what they are asked to study next. VITian Chatbot addresses this gap by combining grounded retrieval with a multi-agent architecture that can research, plan, track, and notify — but this combination also needs to be evaluated on its own terms, since adding agents and retrieval introduces its own costs (latency, token usage, and coordination overhead) that must be justified with evidence rather than assumed.

3.OBJECTIVES

- To design and implement a multi-agent conversational assistant that answers academic, placement, and career-guidance queries using retrieval-augmented generation grounded in verified institutional and company data.
- To build a Supervisor LLM agent using LangGraph that reliably delegates user requests to the correct specialised agent (Company Research, Planner, Progress, Notification).
- To implement an Adaptive Planner Agent that generates personalised study roadmaps and revises them based on observed student progress and struggle signals.
- To automate placement-related notifications using an event-driven pipeline (AWS Lambda, EventBridge, SNS) that parses unstructured emails into structured, actionable alerts.
- To empirically evaluate the system through five targeted analyses covering RAG grounding, Supervisor routing accuracy, multi-agent versus monolithic performance, retrieval quality, and adaptive versus static planning, so that every design claim is backed by measurable evidence rather than assumption.

4.SCOPE OF THE PROJECT

The project is scoped to VIT students preparing for on-campus and off-campus placements, as well as students seeking general academic and career guidance within the university ecosystem. The system covers conversational question answering on company profiles, eligibility criteria, and recruitment processes; personalised study-plan generation and revision; progress tracking against the study plan; and automated notification of placement-related deadlines and events. The system uses placement and academic information collected and structured by the project team as its knowledge base.

In this phase, the project does not cover live human-evaluated mock technical interviews, resume screening by recruiters, or predictive placement-outcome scoring. The factual coverage of the system is also limited to the placement and academic information available in the curated knowledge base during the project timeline and is therefore not intended to function as a general-purpose knowledge engine.

5. PROPOSED SYSTEM

VITian Chatbot is proposed as a supervised multi-agent conversational assistant designed to support students with academic, placement, and career-related queries. Unlike a conventional FAQ chatbot, the system combines natural language understanding, retrieval-augmented generation, multi-agent orchestration, personalised planning, progress tracking, and automated notifications.

A user's message first passes through AWS Lex V2 for natural language understanding. The request is then handled by a Supervisor LLM agent implemented using LangGraph, which classifies the intent and routes the request to the appropriate specialised agent. The Company Research Agent answers placement and company-specific questions using retrieval-augmented generation over the PostgreSQL and ChromaDB knowledge base. The Planner Agent generates personalised study roadmaps based on the student's target companies, current skill level, and available time, while the Progress Agent tracks preparation and provides struggle or mastery signals for adaptive plan revision.

The Notification Agent handles placement-related reminders using information extracted from placement emails. AWS Lambda parses incoming placement emails into structured JSON records stored in Amazon S3, while Amazon EventBridge and SNS support automated reminder delivery. IAM provides access control and CloudWatch provides monitoring and logging. FastAPI acts as the backend layer connecting the conversational interface with the required services and agents.

The main contribution of the proposed system is the integration of grounded retrieval, supervised multi-agent orchestration, progress tracking, and closed-loop adaptive planning within a single placement-preparation assistant. In particular, the Adaptive Planner revises a student's study plan based on observed progress and struggle signals rather than generating a static plan only once. The effectiveness of these components will be evaluated through the five experiments described in the Methodology section.

6.LITERATURE SURVEY

Table 1: Literature Review Of 15 Articles

S.No	Paper Title & Year	Merits	Demerits	Relevance to our project
1	Retrieval-Augmented Generation for Large Language Models: A Survey (Gao et al., 2023)	Consolidates naive, advanced and modular RAG pipelines; compares retrieval, augmentation and generation strategies.	Establishes RAG as a mainstream way to reduce hallucination and ground LLM output in external knowledge.	Forms the conceptual basis for our RAG vs vanilla LLM comparison (Analysis 1) and the modular design of the Company Research Agent.
2	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)	Introduces the original RAG architecture combining a parametric seq2seq generator with a non-parametric dense retriever.	Shows RAG models produce more specific and factual text than parametric-only baselines on open-domain QA.	Provides the theoretical foundation for grounding the Company Research Agent in verified placement/company data.
3	Dense Passage Retrieval for Open-Domain Question Answering (Karpukhin et al., 2020)	Proposes DPR, a dual-encoder dense retriever trained with a small number of question-passage pairs.	Dense retrieval outperforms BM25 by a wide margin on passage retrieval accuracy.	Motivates comparing embedding-based retrieval configurations in our Retrieval Quality Evaluation (Analysis 4).
4	Survey of Hallucination in Natural Language Generation (Ji et al., 2023)	Surveys causes, categories (intrinsic/extrinsic) and metrics for hallucination across NLG tasks.	Gives a structured vocabulary and metric families for measuring hallucination, useful for human annotation protocols.	Directly informs how we define and annotate the Hallucination Rate metric used in Analysis 1.
5	RAGAs: Automated Evaluation of Retrieval Augmented Generation (Es et al., 2024)	Proposes a reference-free evaluation framework with metrics such as Faithfulness, Answer Relevance and Context Relevance.	Enables RAG evaluation without expensive human-labelled ground truth for every query.	Guides our combination of automated and human-annotated evaluation for the RAG vs vanilla LLM study.

6	Large Language Model based Multi-Agents: A Survey of Progress and Challenges (Guo et al., 2024)	Reviews architectures, communication protocols and coordination strategies for LLM-based multi-agent systems.	Documents how task decomposition across specialised agents can improve accuracy over a single monolithic LLM.	Directly supports our Multi-Agent vs Monolithic ablation study (Analysis 3) and the Supervisor-agent design.
7	The Rise and Potential of Large Language Model Based Agents: A Survey (Xi et al., 2023)	Surveys the agent construction pipeline (brain, perception, action) and single- vs multi-agent orchestration patterns.	Frames the Supervisor/delegation pattern used by orchestration frameworks such as LangGraph.	Supports the design rationale for the Supervisor LLM's routing behaviour, evaluated in Analysis 2.
8	Implementing Multi-Agent Systems using LangGraph: A Comprehensive Study (Biju, 2026)	Studies LangGraph's graph-based orchestration model for building stateful multi-agent LLM applications.	Demonstrates how LangGraph nodes and conditional edges can implement supervisor-style routing between agents.	Provides the direct technical basis for our Supervisor-based routing implementation, evaluated in Analysis 2.
9	Large Language Model Based Educational Virtual Assistant using RAG Framework (Khan et al., 2025)	Builds a RAG-based educational virtual assistant and evaluates its answer quality against a plain LLM baseline.	Reports that grounding responses in institutional documents improves answer relevance and reduces incorrect answers.	Closely parallels our RAG vs vanilla LLM comparison, but applied to placement/company data rather than general academics.
10	From Questions to Insightful Answers: Building an Informed Chatbot for University Resources (Neupane et al., 2024)	Develops a RAG-based university chatbot that answers student queries from institutional knowledge sources.	Shows that retrieval grounding reduces incorrect or fabricated answers about university-specific information.	Highlights the coverage gap that VITian Chatbot addresses by adding planning, tracking and notification agents.
11	AdaptJobRec: Enhancing Conversational Career Recommendation through an LLM-Powered Agentic System (Wang et al., 2026)	Proposes an agentic conversational system for career recommendation using an LLM-powered planning loop.	Demonstrates that agentic planning can personalise career guidance better than a single-turn recommender.	Supports the design of the Adaptive Planner Agent and its closed-loop replanning behaviour (Analysis 5).

12	A New Paradigm of Personalized Education Driven by Multi-Agent Collaboration (Zhao, 2025)	Discusses how collaborating specialised agents can personalise instructional content and pacing for learners.	Argues that agent specialisation and coordination improve adaptability compared with single-model tutoring.	Provides conceptual grounding for treating adaptive planning as a distinct, separately measurable capability.
13	Artificial Intelligence in Intelligent Tutoring Systems toward Sustainable Education: A Systematic Review (Lin, Huang, & Lu, 2023)	Systematically reviews AI-based intelligent tutoring systems and the techniques used to adapt to learner needs.	Confirms that adaptive, feedback-driven tutoring generally outperforms static, one-size-fits-all instruction.	Supports comparing the Adaptive Planner against a static study plan as a legitimate, literature-backed evaluation design.
14	Building a Legal Dialogue System: Development Process, Challenges and Opportunities (Sharma et al., 2021)	Documents the end-to-end design of a domain-specific dialogue system, including intent classification and routing.	Offers practical lessons on evaluating intent/routing accuracy and handling out-of-scope queries in dialogue systems.	Informs the routing-accuracy evaluation methodology used for the Supervisor agent in Analysis 2.
15	Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (Zheng et al., 2023)	Studies the reliability of using strong LLMs as automated judges for open-ended chatbot responses.	Shows LLM judges can align closely with human preference when biases such as position and verbosity are controlled.	Guides the rubric-based human-plus-LLM scoring approach used for Plan Quality in Analysis 3.

6.1 Findings in literature Survey

Across the papers reviewed, retrieval-augmented generation is consistently reported to reduce hallucination and improve factual grounding compared with relying on an LLM's parametric memory alone, but almost all of this evidence comes from open-domain or general-purpose settings rather than a fast-changing, narrow domain like campus placements. This is a gap our VITian Chatbot is attempting to address, by applying RAG specifically to placement and company data, where facts change every recruitment season and there is little room for a confidently wrong answer.

The multi-agent LLM literature makes a strong conceptual case that decomposing a task across specialised agents improves handling of complex, multi-step workflows, but there is limited focus on quantifying whether this holds up against a single, well-prompted LLM doing the same job end to end. Our Multi-Agent vs Monolithic ablation study is designed to check exactly this, rather than assume multi-agent decomposition is automatically better.

On the retrieval side, the literature is clear that retrieval quality — chunking strategy, embedding choice, retriever architecture — has a direct effect on downstream answer quality. Because of this, retrieval quality needs to be evaluated independently of the final chatbot response rather than

assumed to be adequate simply because a RAG pipeline is in place.

Finally, the intelligent-tutoring and adaptive-learning literature supports the idea that closed-loop, feedback-driven instruction generally outperforms static, one-size-fits-all plans, but this evidence is drawn mostly from classroom and general tutoring contexts rather than placement preparation specifically. This project attempts to address that gap by treating the Adaptive Planner's benefit over a static study plan as an experimental question rather than a design assumption, since this is the project's strongest novelty claim and the one that most needs objective evidence before it can be called superior.

7.METHODOLOGY

The overall methodology follows a standard multi-agent, retrieval-augmented pipeline: user intent is captured by AWS Lex V2, routed by a LangGraph Supervisor to the appropriate specialised agent, and answered either through retrieval-augmented generation (Company Research Agent) or through stateful planning and tracking (Planner and Progress Agents), with the Notification Agent closing the loop through automated reminders. Rather than assuming this design is correct simply because it is more elaborate than a single-prompt chatbot, we treat every major architectural decision as a hypothesis to be tested. The five analyses below are placed under Methodology because they are not an afterthought evaluation bolted onto a finished system; they define how each subsystem (retrieval, routing, agent decomposition, and adaptive planning) will be built, instrumented, and judged as the project proceeds from Review-1 to the working prototype.

Each analysis below follows the same structure: objective, experimental setup, evaluation metrics, hypothesis, expected outcome, significance, limitations, and a suggested result table with placeholder (—) values to be filled in once the experiment is actually run. None of the numbers shown at this stage are real results; they are hypotheses stated up front so that the eventual results can be compared honestly against what we expected.

7.1 Analysis 1: RAG Grounding vs Vanilla LLM (Hallucination Reduction)

Objective

This experiment checks whether grounding the Company Research Agent's answers in verified, retrieved company and placement data actually reduces hallucination compared with letting a raw, ungrounded LLM (GPT or Gemini) answer the same questions from its parametric memory alone.

Experimental Setup

The two systems being compared are the RAG-grounded Company Research Agent (proposed system) and a raw GPT/Gemini instance answering the same prompts with no retrieval access (baseline).

A query set of company-specific placement questions will be prepared, covering eligibility criteria, recruitment process steps, role descriptions, and past interview patterns for a defined set of recruiting companies.

Both systems receive the identical set of questions under identical prompting conditions, so that any difference in answer quality can be attributed to grounding rather than prompt wording.

Each answer is checked against the verified source document it should have been grounded in, using a scoring rubric agreed upon with the guide, and a subset is cross-checked by a second annotator to gauge agreement.

The exact number of test questions is a proposed experimental setup and will be finalised once the company knowledge base is built; it is not fixed at this review stage.

Evaluation Metrics

- Factual Accuracy (%): the proportion of answered questions where the stated facts (dates, eligibility numbers, process steps) match the verified source document.
- Hallucination Rate (%): the proportion of answers containing at least one fabricated or unsupported claim, based on human annotation against the source documents.

- Citation Precision (%): for the RAG system, the proportion of cited source passages that actually support the claim they are attached to.

Hypothesis

It is hypothesized that the RAG-grounded Company Research Agent will achieve more than 90% factual accuracy on company-specific facts, while the ungrounded LLM baseline will perform below 60% on the same questions. These figures are proposed hypotheses only and are not being presented as measured results at this stage.

Expected Outcome

A result supporting the project's grounding claim would show the RAG-grounded agent scoring meaningfully higher on factual accuracy and citation precision, and meaningfully lower on hallucination rate, than the ungrounded baseline on the same question set.

Significance to the Project

This experiment gives direct, numeric evidence for the project's central claim in Section 5 that grounding responses in verified company information reduces hallucination, rather than leaving that claim as an unverified design assertion.

Limitations

- Human annotation of factual accuracy and hallucination is inherently somewhat subjective and depends on annotator familiarity with placement processes.
- Results depend on the specific LLM/version used for both the grounded and ungrounded systems and may not generalise to other model versions.
- The company knowledge base is limited to what the team is able to collect during the project timeline, which bounds the range of questions that can be tested.
- A small evaluation set may not capture rarer or more ambiguous question types that could behave differently.

Suggested Result Table

Metric	Proposed System	Baseline	Improvement
Factual Accuracy (%)	—	—	—
Hallucination Rate (%)	—	—	—
Citation Precision (%)	—	—	—

7.2 Analysis 2: Supervisor Routing Accuracy

Objective

This experiment checks whether the LangGraph Supervisor's LLM-based intent classification routes user requests to the correct specialised agent more reliably than a simple rule-based or keyword-matching router.

Experimental Setup

The two systems compared are the LangGraph Supervisor (proposed system) and a simple rule-based/keyword router (baseline) built to handle the same four agents: Company Research, Planner, Progress, and Notification.

A labelled set of user utterances will be prepared, each tagged with the agent it should be routed to, including some deliberately ambiguous or multi-intent utterances to stress-test both routers.

Both routers receive the same utterance set, and their routing decisions are compared against the ground-truth labels.

Latency is measured as the additional time the Supervisor's LLM call adds compared with the near-instant keyword baseline, under comparable test conditions.

The exact size of the labelled utterance set is a proposed setup detail and will be finalised alongside the guide once the agent boundaries are frozen.

Evaluation Metrics

- Routing Precision (per agent): of all utterances routed to a given agent, the proportion that actually belonged there.
- Routing Recall (per agent): of all utterances that truly belonged to a given agent, the proportion the router correctly sent there.
- Overall Routing Accuracy: the proportion of all utterances routed to the correct agent across the whole test set.
- Misroute Rate: the proportion of utterances sent to the wrong agent, reported separately since a misroute is more costly than a correct-but-imprecise flag.
- Additional Latency: the extra response time introduced by the Supervisor's routing hop compared with the keyword baseline.

Hypothesis

It is expected that the Supervisor will achieve around 95% or higher overall routing accuracy, compared with approximately 75% for the keyword-based baseline. These are proposed hypotheses, not measured results, and are stated here only to be tested against actual data later.

Expected Outcome

A result supporting the project's delegation claim would show the Supervisor achieving noticeably higher precision, recall, and overall accuracy than the keyword baseline, at an added latency small enough to remain acceptable for a conversational assistant.

Significance to the Project

This experiment quantifies the "intelligent delegation" novelty claim with actual precision, recall, and accuracy numbers instead of leaving it as an assumed benefit of using an LLM-based router.

Limitations

- The keyword-based baseline's performance depends heavily on how carefully its rules are written, so a weak baseline could overstate the Supervisor's advantage.
- Ambiguous, multi-intent utterances may not have a single unambiguous ground-truth label, introducing some annotation subjectivity.
- Latency measurements can vary with model load, network conditions, and hardware, so reported numbers should be treated as indicative rather than absolute.
- The labelled utterance set will be limited in size and may not cover every phrasing style students actually use.

Suggested Result Table

Metric	Proposed System	Baseline	Improvement
Overall Routing Accuracy (%)	—	—	—
Misroute Rate (%)	—	—	—
Added Latency (ms)	—	—	—

7.3 Analysis 3: Multi-Agent vs Monolithic LLM Pipeline (Ablation Study)

Objective

This experiment checks whether decomposing the system into specialised agents provides a measurable benefit over asking a single, monolithic LLM prompt to perform the same range of tasks end to end.

Experimental Setup

The two systems compared are the full multi-agent VITian Chatbot (proposed system) and a single-prompt LLM pipeline instructed to perform company research, planning, and progress tracking within one prompt (baseline).

A shared set of representative user sessions will be used, covering a mix of company-research questions, plan-generation requests, and progress-check-ins, so both systems are tested on comparable workloads.

Both systems process the same sessions, and their outputs are scored using a predefined rubric covering task completion, correctness, and coherence, in addition to logging latency and token usage.

This is an ablation study in the strict sense: the multi-agent system is not compared against an unrelated product, but against a reduced version of itself with the agent boundaries collapsed into one prompt.

The number of test sessions is a proposed setup detail, to be fixed once representative session scripts are drafted with the guide.

Evaluation Metrics

- Task Completion Accuracy: the proportion of sessions where the system produced a correct, usable result for the requested task.
- Response Latency: the end-to-end time taken to produce a final answer, including any agent-to-agent hops in the multi-agent system.
- Token Cost per Session: the total number of LLM tokens consumed to complete a session, used as a proxy for compute/API cost.
- Plan Quality: a rubric score (for study-plan-related sessions) assessing whether the generated plan is realistic, relevant to the target companies, and appropriately sequenced.
- Overall Response Quality: a rubric-based score combining correctness, coherence, and helpfulness, scored independently by a human rater and cross-checked where feasible.

Hypothesis

It is hypothesized that the multi-agent pipeline will improve task completion accuracy and plan quality over the monolithic baseline, at the cost of some additional latency and token usage from agent coordination. No specific percentage improvement is claimed at this stage; the experiment is designed to measure this trade-off objectively rather than assume a particular outcome.

Expected Outcome

A result supporting the project's architectural claim would show the multi-agent system scoring higher on task completion accuracy and plan quality, in a way that reasonably justifies whatever additional latency and token cost the coordination overhead introduces. If the monolithic baseline performs comparably at lower cost, that would be an equally valid and important finding.

Significance to the Project

This is the core defence for the project's architectural decomposition: it proves, or disproves, that splitting the system into specialised agents adds measurable value beyond simply prompting a general-purpose LLM to do everything.

Limitations

- Rubric-based scoring of plan quality and overall response quality carries some subjectivity even with a predefined rubric and a second rater.
- Token cost and latency can vary with model version, provider load, and prompt length, so absolute numbers may shift between runs.
- The monolithic baseline's performance depends on how well its single prompt is engineered, and a poorly written baseline prompt could unfairly favour the multi-agent system.
- A limited number of test sessions may not cover every workflow combination a real user might attempt.

Suggested Result Table

Metric	Proposed System	Baseline	Improvement
Task Completion Accuracy (%)	—	—	—
Response Latency (s)	—	—	—
Token Cost per Session	—	—	—
Plan Quality (rubric score)	—	—	—

7.4 Analysis 4: Retrieval Quality Evaluation

Objective

This experiment checks whether the retrieval component of the RAG pipeline is actually surfacing the most relevant information before the LLM generates an answer, independent of how good the final generated text sounds.

Experimental Setup

A manually prepared and labelled query set will be built around placement, companies, eligibility, recruitment processes, and interview/aptitude topics, each query paired with the passages in the knowledge base that should be retrieved for it.

Different chunking strategies (fixed-size chunking vs semantic chunking) and, where feasible, different embedding models will be compared on the same query set and the same underlying document collection.

Retrieval is scored purely on whether the correct passages are returned and how highly they are ranked, without involving the LLM's generated answer at this stage, so that retrieval quality is measured independently of generation quality.

The exact number of labelled queries is a proposed setup detail and will depend on how much of the knowledge base has been curated by the time this experiment is run.

Evaluation Metrics

- Precision@5: of the top 5 retrieved passages, the proportion that are actually relevant to the query.
- Recall@5: of all the passages that are relevant to the query, the proportion that appear within the top 5 retrieved results.
- Mean Reciprocal Rank (MRR): the average, across all queries, of one divided by the rank position of the first relevant passage retrieved; higher MRR means relevant passages appear earlier.
- nDCG@10 (Normalized Discounted Cumulative Gain): a ranking-quality score over the top 10 results that rewards relevant passages more when they appear earlier in the list, normalised against an ideal ranking.

Hypothesis

It is hypothesized that semantic, domain-appropriate chunking combined with a suitable embedding model will outperform naive fixed-size chunking, potentially improving nDCG@10 by roughly 15–20%. This figure is a proposed hypothesis only and must not be treated as an actual measured result until the experiment is run.

Expected Outcome

A result supporting this part of the design would show the semantic-chunking configuration achieving higher Precision@5, Recall@5, MRR, and nDCG@10 than the fixed-size-chunking baseline on the same labelled query set.

Significance to the Project

This experiment demonstrates that the retrieval component genuinely finds relevant information before the LLM ever generates an answer, which is the standard, reviewer-expected way to validate a RAG pipeline and strengthens confidence in the Analysis 1 results as well.

Limitations

- A manually labelled query set is inherently limited in size and may not represent the full diversity of real student queries.
- Relevance labelling itself involves some human judgement and can vary between annotators.
- Results are specific to the embedding models and chunking configurations tested and may not generalise to future model or corpus changes.
- The knowledge base used for this evaluation is limited to what has been collected and curated at this stage of the project.

Suggested Result Table

Metric	Proposed System	Baseline	Improvement
Precision@5	—	—	—
Recall@5	—	—	—
MRR	—	—	—
nDCG@10	—	—	—

7.5 Analysis 5: Adaptive Planner Effectiveness (Closed-Loop vs Static Planning)

Objective

This experiment checks whether the Adaptive Planner Agent, which revises a student's study plan based on observed progress and struggle signals, provides a measurable benefit over a static study plan that is generated once and never revised.

Experimental Setup

The two systems compared are the Adaptive Planner (proposed system), which observes progress signals and replans, and a Static Study Plan (baseline) generated once at the start and left unchanged.

The study can use real student interaction data if it becomes available during the project timeline, or a controlled simulated student dataset if real usage data is not yet available; whichever is used will be clearly labelled as such in the eventual results, and simulated data will never be presented as real student evidence.

Both the adaptive and static conditions will be run over a comparable time window and comparable set of target topics, so that any observed difference can be attributed to the planning strategy rather than to unequal study time.

Replanning events triggered by the Adaptive Planner will be logged and reviewed to check whether they occurred in response to genuine struggle signals rather than arbitrarily.

Evaluation Metrics

- Topic Mastery Rate: the proportion of targeted topics a student reaches an acceptable competency level in, by the end of the study window.
- Time-to-Competency: how long it takes, on average, for a student to reach acceptable competency on a topic.
- Problem-Solving Success Rate: the proportion of practice problems solved correctly after working through a given topic in the plan.
- Improvement in Weak Topics: the change in performance specifically on topics that were initially flagged as weak, before versus after the study period.
- Number/Effectiveness of Replanning Events: how often the Adaptive Planner revised the plan, and whether those revisions were followed by measurable improvement.

Hypothesis

It is hypothesized that closed-loop adaptive replanning will improve weak-topic mastery over a static plan across the same study time window. No specific improvement percentage is claimed at this stage, since this is the project's strongest novelty claim and needs to be evaluated carefully and objectively rather than assumed to succeed.

Expected Outcome

A result supporting this claim would show students under the Adaptive Planner reaching higher topic mastery, better problem-solving success, and stronger improvement on weak topics than students following a static plan over the same time window, with replanning events plausibly linked to those gains.

Limitations

- If simulated student data is used, it cannot fully capture the variability and unpredictability of real student behaviour and must be clearly labelled as simulated, not real, evidence.

- Real student participation, if used instead, is likely to involve a small sample size and a short observation window given the project timeline.
- Mastery and competency measurements depend on how test/assessment items are constructed, which introduces some subjectivity.
- Replanning effectiveness is hard to isolate from other factors such as a student's own motivation or external study effort during the observation window.

Suggested Result Table

Metric	Proposed System	Baseline	Improvement
Topic Mastery Rate (%)	—	—	—
Time-to-Competency	—	—	—
Problem-Solving Success Rate (%)	—	—	—
Improvement in Weak Topics (%)	—	—	—

8. SOFTWARE REQUIREMENTS

8.1 Functional Requirements

- The system shall accept natural-language student queries through a conversational interface and interpret intent using AWS Lex V2.
- The system shall route each interpreted request to the correct specialised agent (Company Research, Planner, Progress, Notification) via the LangGraph Supervisor.
- The Company Research Agent shall answer placement and company-related queries using retrieval-augmented generation over the PostgreSQL/ChromaDB knowledge base, with citations to source documents where applicable.
- The Planner Agent shall generate a personalised study roadmap based on a student's target companies, current skill level, and available time, and shall revise the roadmap when the Progress Agent flags a struggle signal.
- The Progress Agent shall track a student's completion status and performance against the current study plan and shall pass struggle/mastery signals to the Planner Agent.
- The Notification Agent shall generate and dispatch reminders for placement-related deadlines and events using data parsed from placement emails.
- The system shall parse incoming placement emails into structured JSON records via AWS Lambda and store them in Amazon S3 for downstream use.

8.2 Non-Functional Requirements

- Reliability: the system should degrade gracefully (e.g., fall back to a safe default response) when a downstream agent or the retrieval store is unavailable.
- Latency: conversational responses should be returned within an interactive time budget acceptable for a chat interface, even with the added Supervisor routing hop.
- Security: access between services shall be scoped through IAM roles following the principle of least privilege, and sensitive student data shall be protected in transit and at rest.
- Observability: CloudWatch logging and monitoring shall be enabled across the Lambda, FastAPI, and agent layers to support debugging and the latency/accuracy measurements needed for the experiments in Section 7.
- Scalability: the architecture should tolerate a growing number of concurrent student sessions without a redesign, given the serverless components (Lambda, EventBridge, SNS) used in the pipeline.
- Maintainability: agents shall be modular enough that a single agent's prompt or logic can be updated without requiring changes to the other agents.

9.SYSTEM ARCHITECTURE

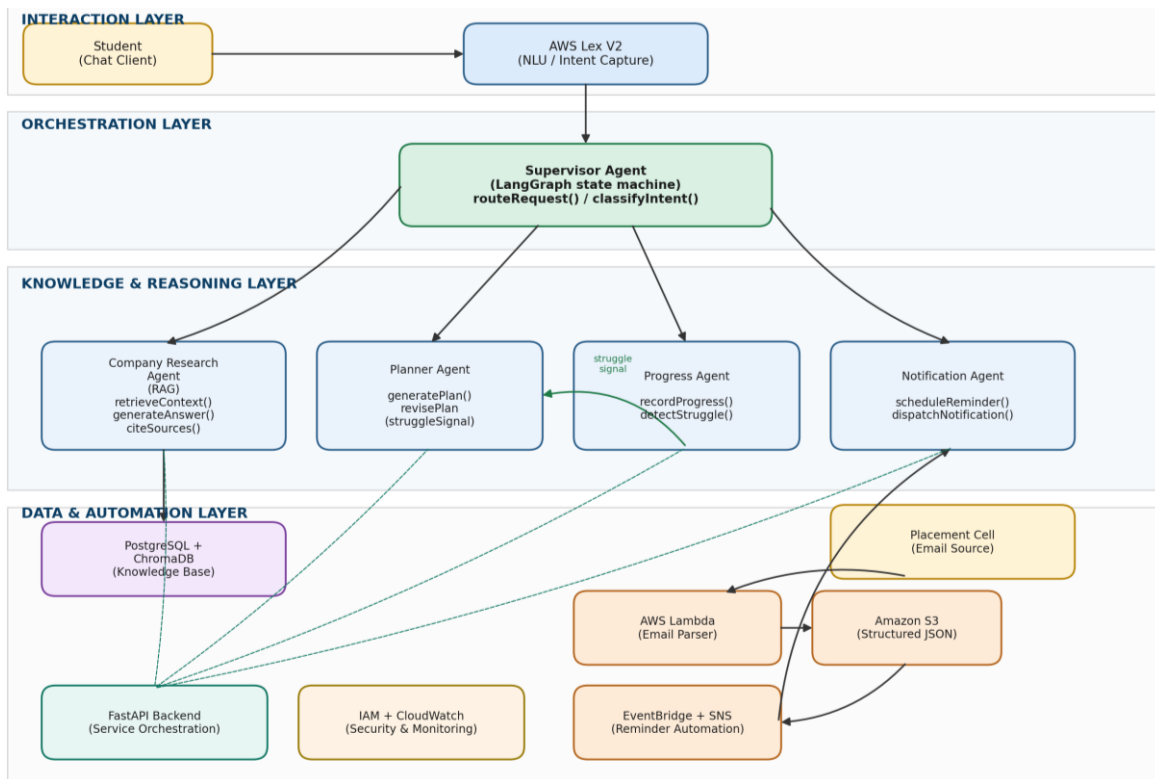


Fig 9.1: Architecture Diagram Of The System

The architecture is organised into four broad layers, described below.

- Interaction layer: AWS Lex V2 handles natural language understanding for incoming student messages before they reach the Supervisor.
- Orchestration layer: the LangGraph Supervisor LLM agent routes each request to the Company Research, Planner, Progress, or Notification agent based on classified intent.
- Knowledge and reasoning layer: the Company Research Agent performs retrieval-augmented generation over a PostgreSQL and ChromaDB knowledge base; the Planner and Progress agents maintain and update each student's study state.
- Data and automation layer: AWS Lambda parses placement emails into structured JSON stored in Amazon S3; Amazon EventBridge and SNS automate reminder delivery; IAM and CloudWatch secure and monitor the whole pipeline; FastAPI exposes these services to the conversational front end.

10.UML DIAGRAMS

The UML diagrams below represent the system's primary actors, use cases, classes, and interaction flow.

Use Case Outline

- Actors: Student (primary), Placement Cell / Email Source (secondary, via automated email ingestion), Guide/Admin (secondary, for monitoring).
- Key use cases for Student: Ask Company/Placement Question, Request Study Plan, View/Update Progress, Receive Notification.
- Key use cases involving the system internally: Route Request (Supervisor), Retrieve Grounded Answer (Company Research Agent), Generate/Revise Plan (Planner Agent), Detect Struggle Signal (Progress Agent), Parse Placement Email (Lambda), Dispatch Reminder (Notification Agent).

Class Outline

- Student: studentId, targetCompanies, skillProfile, progressHistory.
- SupervisorAgent: routeRequest(), classifyIntent().
- CompanyResearchAgent: retrieveContext(), generateAnswer(), citeSources().
- PlannerAgent: generatePlan(), revisePlan(struggleSignal).
- ProgressAgent: recordProgress(), detectStruggle().
- NotificationAgent: scheduleReminder(), dispatchNotification().
- PlacementEmailParser: parseEmail(), toStructuredRecord().
- KnowledgeBase: storeDocument(), retrieveTopK(query).

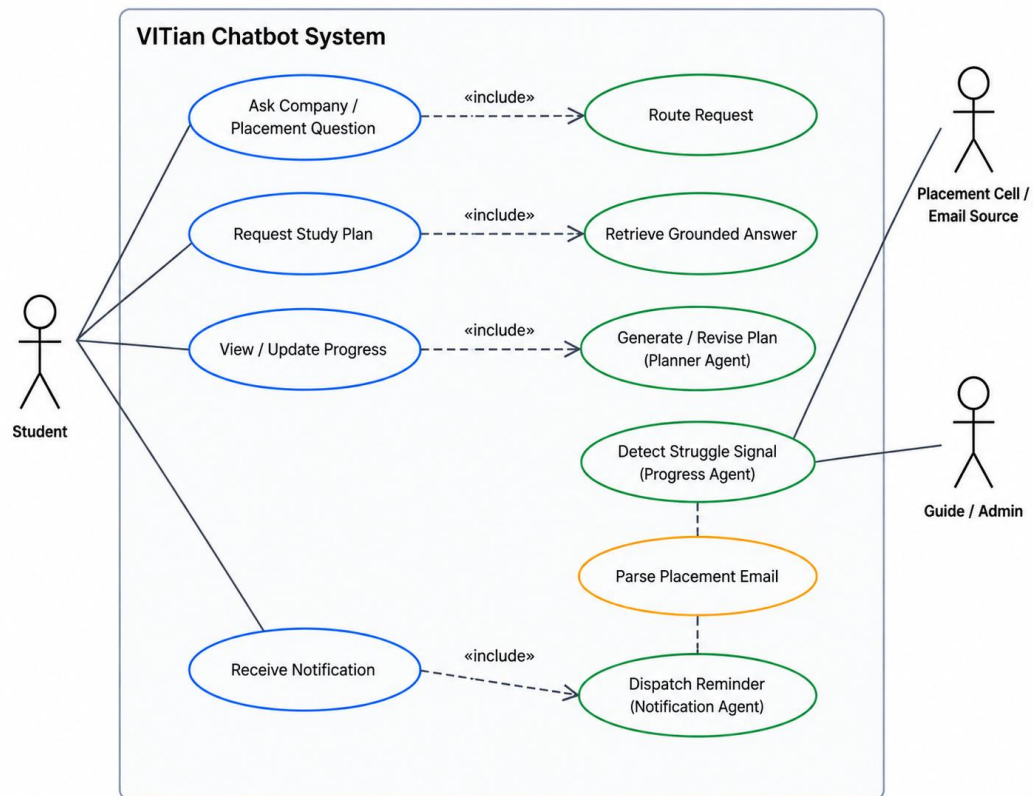


Fig 10.1: Use Case Diagram – VITian Chatbot

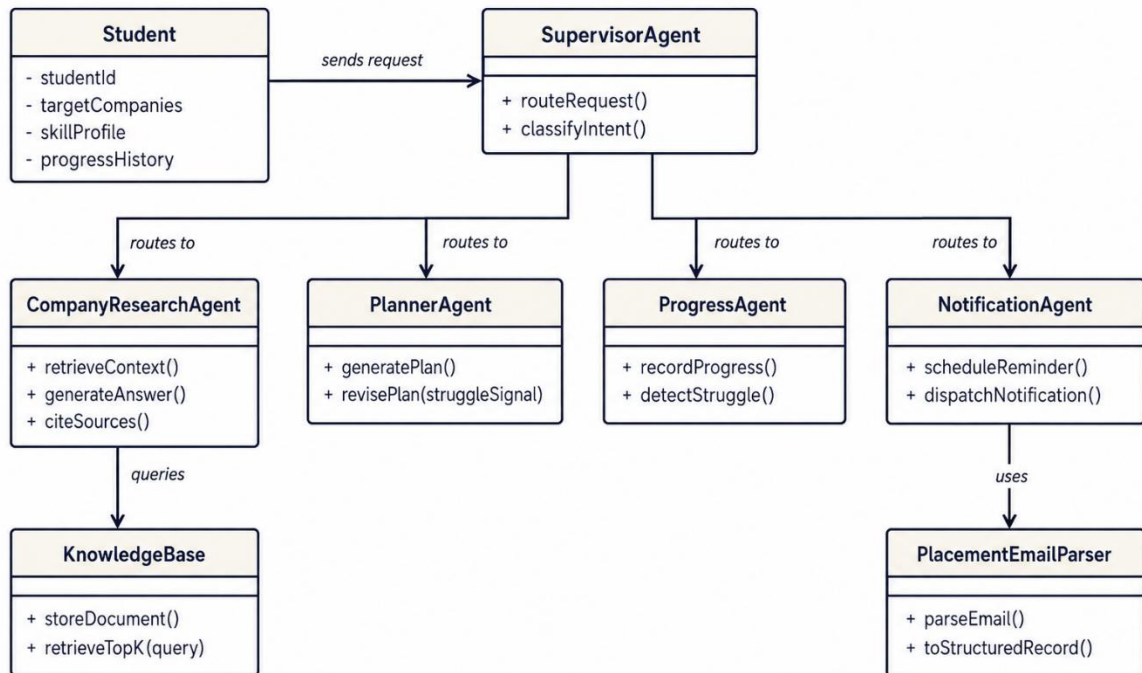


Fig 10.2: Class Diagram – VITian Chatbot

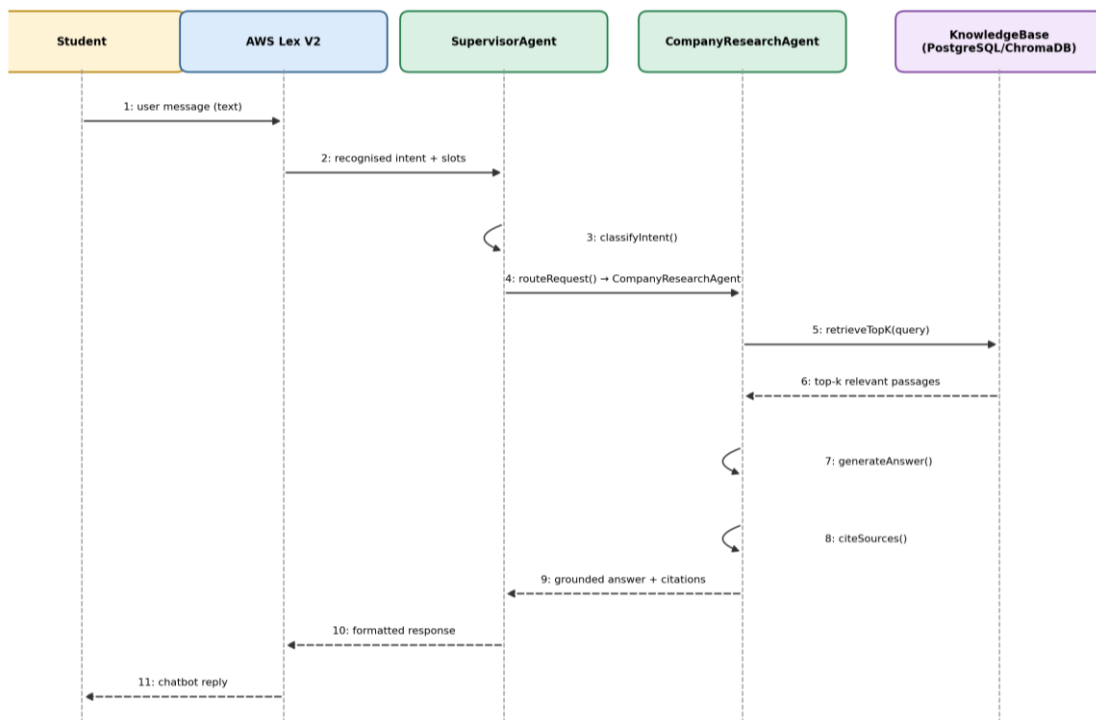


Fig 10.3: Sequence Diagram – “Ask Company/Placement Question” Flow

11.SUMMARY

VITian Chatbot proposes a multi-agent, retrieval-augmented conversational assistant that goes beyond a static FAQ bot by grounding its answers in verified placement data and by closing the loop between planning, progress, and notification. This Review-1 report has set out the problem being addressed, the literature that motivates and constrains the design, the proposed system and its methodology, and, most importantly, five concrete experiments that will test the project's core claims: that grounding reduces hallucination, that Supervisor-based routing delegates more accurately than keyword matching, that multi-agent decomposition is worth its overhead compared with a monolithic LLM, that the retrieval layer is actually surfacing relevant content, and that closed-loop adaptive planning outperforms a static study plan. No experimental results are reported at this stage; the numbers shown throughout this document are hypotheses, not findings, and the corresponding tables have been left blank for results to be filled in as the experiments are carried out in the next phase of the project.

12. REFERENCES

1. Biju, S. M. (2026). Implementing multi-agent systems using LangGraph: A comprehensive study. In R. Nayak, N. Mittal, A. Khunteta, & M. Kumar (Eds.), *Recent advancements in artificial intelligence (ICRAAI 2025)*, Lecture Notes in Networks and Systems (Vol. 1468). Springer.
2. Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
3. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv*. <https://arxiv.org/abs/2312.10997>
4. Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)* (pp. 8048–8057).
5. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), Article 248. <https://doi.org/10.1145/3571730>
6. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 6769–6781). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
7. Khan, U. H., Khan, M. H., & Ali, R. (2025). Large language model based educational virtual assistant using RAG framework. *Procedia Computer Science*, 252, 905–911.
8. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
9. Lin, C. C., Huang, A. Y., & Lu, O. H. (2023). Artificial intelligence in intelligent tutoring systems toward sustainable education: A systematic review. *Smart Learning Environments*, 10(1), Article 41.
10. Neupane, S., Hossain, E., Keith, J., & Rahimi, S. (2024). From questions to insightful answers: Building an informed chatbot for university resources. *arXiv*. <https://arxiv.org/abs/2405.08120>
11. Sharma, M., Russell-Rose, T., Barakat, L., & Matsuo, A. (2021). Building a legal dialogue system: Development process, challenges and opportunities. *arXiv*. <https://arxiv.org/abs/2109.00381>
12. Wang, Q., Wang, D., Chen, K., Hu, Y., Girdhar, P., Wang, R., & Wu, X. (2026). AdaptJobRec: Enhancing conversational career recommendation through an LLM-powered agentic system. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(47), 40473–40479.
13. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al. (2023). The rise and potential of large language model based agents: A survey. *arXiv*. <https://arxiv.org/abs/2309.07864>
14. Zhao, Z. (2025). A new paradigm of personalized education driven by multi-agent collaboration. *Academic Journal of Sociology and Management*, 3(3), 9–17.

15. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36, 46595–46623.